

PREDICTIONS ON WINE QUALITY DATA

Valeria Riccardo

Data Science for Economics

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Abstract	2
2	Theoretical background	2
3	Linear predictors	3
3.1	Logistic Regression	3
3.1.1	Key functions	4
3.2	Support Vector Machine	6
3.2.1	Key functions	6
4	Evaluation Metrics	9
5	Hyperparameter tuning	10
5.1	Stratified Cross Validation	10
5.2	GridSearch	10
6	Data Exploration	11
7	Models fitting and results	12
7.1	Data preprocessing	12
7.2	Logistic regression results	12
7.3	Support Vector Machine results	13
7.4	Predictions on new data and conclusions	13

1 Abstract

This project investigates the prediction of wine quality using machine learning models. We address class imbalance through oversampling and explore both logistic regression and Support Vector Machine models. Feature selection and hyperparameter tuning are applied to optimize model performance, and correlation analysis is used to support variable choice. The study highlights the importance of kernel selection, parameter tuning, and model interpretability in developing robust predictive models.

2 Theoretical background

Classification problems involve learning a function that maps input features to a discrete output, which consists of a set of categories or classes. Unlike regression problems, where the target variable Y is continuous, making the relationship between the target and explanatory variables smoother, the loss function (Mean Squared Error, MSE) is easier to optimize and there is no strict decision boundary since the target values lie along a continuous real line. In contrast, a discrete target Y introduces challenges such as nonlinear decision boundaries, more complex loss functions, and more sophisticated models to accurately classify data.

The *Bayes optimal classifier* serves as the theoretical benchmark for classification problems, as it provides the theoretical foundation to minimize the probability of error by always predicting the class with the highest posterior probability, $P(Y | X)$.

$$P(Y | X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

Where $P(X | Y)$ is the likelihood (the probability of observing X given a class Y), $P(Y)$ is the prior probability of Y and $P(X)$ is the marginal probability of the features (the probability of observing a certain combination of features).

In relation to our dataset, which contains information about wine quality, $P(X | Y)$: for a certain wine quality (1 for high-quality wines, 0 for low-quality wines), I observe certain values of X ;

$P(Y)$: before analyzing the wine features, we analyze the wine quality distribution.

$P(X)$: before considering the quality of the wine, what is the probability of observing a specific combination of characteristics of the wine?

The Bayes optimal classifier assumes that we know the true data distribution, which does not always align with reality. For this reason, we can only approximate the Bayes classifier as closely as possible, even though we cannot precisely compute $P(Y | X)$.

In this context, we limit the choice to linear predictors, specifically Logistic Regression and Support Vector Machines: linear predictors typically provide good generalizations and are fast and easy to train.

The linear predictors aim at finding a classifier such that $h(x) = f(w^T x)$ which minimizes the expected loss (or risk) over the training dataset. The loss aims to

minimize the number of misclassifications. The formulation of the loss function, using the 0-1 loss, is NP-hard meaning that it will lead to a combinatorial optimization problem (we should analyze all the possible configurations of w and b to minimize misclassification) that is not scalable and is computationally expensive.

The solution is to apply a convex surrogate loss, a convex and differentiable function, such as the log loss or hinge loss. Having a convex loss function leads to many advantages: first of all, we always have a global minimum; secondly, it is always differentiable, allowing the application of optimization algorithms such as gradient descent (which consists of updating the weights in the direction of the negative gradient of the loss function).

3 Linear predictors

- W is the weight vector with n features, a set of parameters that determines the orientation and position of the decision boundary separating the classes in the hyperplane.

- X is the input feature vector, which defines the hyperplane dimensions. The weight of each feature determines its importance in the separation process.

- b is the bias term, which shifts the decision boundary and determines where the hyperplane intersects the y-axis.

The goal of a classification problem is to place the X values in the correct region of a hyperplane, a geometric object that divides the space into half-spaces (regions). The features are represented as points in the space, and they are classified based on the result of $w^T x + b$: if the sign of p is positive, we assign $Y = 1$, if it is negative, we assign $Y = 0$.

However, the optimization of the loss function can lead to overfitting: the model fits the data too strictly and loses its ability to generalize to new data. To address this, we introduce a hyperparameter, commonly referred to as λ for logistic regression, and C for SVMs, which helps balance the bias-variance tradeoff. If the model is too complex, the variance will be high, making the model overly sensitive to fluctuations in the data. On the other hand, too much bias will prevent the model from generalizing well. By introducing a regularization parameter, we assume a prior distribution on the weights (Gaussian if the penalty is L2, and Laplace if the penalty is L1), each with distinct advantages. When applying L2 regularization, we raise the weights to the second power, pushing them toward lower values symmetrically. In contrast, L1 regularization uses the norm of the absolute values of the weights, forcing some weights to become zero, which also allows for feature selection.

3.1 Logistic Regression

The term "regression" is employed because logistic regression can be viewed as an extension of the regression models, sharing the same mathematical formulation $y = \beta x + errors$, where β is replaced by the model weights, with an

additional bias term. However, rather than modeling the marginal effect on a continuous target variable, logistic regression models the probability that y belongs to a particular class. This is achieved by applying the logit function (or sigmoid function) to the linear combination of features.

In the implementation, the logistic regression model is encapsulated in a `LogRegression` object, with the following parameters:

- alpha*: learning rate (default 0.001).
- lambda_reg*: regularization parameter, with `_reg` added to avoid conflict with Python's `lambda`. Default is 0 (no regularization).
- penalty*: specifies the type of regularization (L2 default, or L1, elasticnet). It is important to highlight that this parameter activates only if *lambda_reg* > 0.
- epochs*: number of gradient descent iterations.
- early_stopping*: boolean flag for stopping early based on validation loss.
- max_iter*: maximum iterations without improvement for early stopping.
- verbose*: prints training information if True.

3.1.1 Key functions

-*Sigmoid*: The logistic function transforms the linear combination of features into a probability in the interval $[0, 1]$. The sigmoid function chosen (expit) belongs to the package *scipy.special* and it is designed to handle eventual numerical overflows.

-*Loss*: The log loss function is defined as:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Within the code, we first apply *np.clip* as a safeguard to avoid computing $\log(0)$, which would otherwise result in invalid (NaN) values. Then, we reproduce the log-loss formula using NumPy methods, and extend it if a regularization penalty is applied.

The *ValueError* acts as a safety check to prevent invalid inputs for the penalty parameter.

-*Gradient computing*: Training is formulated as an optimization problem, where the objective is to find the parameters minimizing the loss. The gradients of the weights and bias are computed as:

$$\nabla_w = \frac{1}{n} X^\top (\hat{y} - y)$$

$$\nabla_b = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

and adjusted for regularization if applicable. In the implementation, this corresponds to updating *gradient_w*. Full-batch gradient descent is implemented, providing stable convergence to the global minimum despite higher computational cost.

-*Fit function*: the fit method accepts training features and target, shuffles the dataset to prevent learning order patterns, and optionally creates a validation set for early stopping. Parameters are updated iteratively according to the gradient and learning rate. After training, the best parameters are saved and inferential statistics including standard errors, z-scores, p-values, and Akaike Information Criterion (AIC) are computed.

The standard errors of each estimated coefficient measures the expected variability of the estimate across repeated samples. In logistic regression, the SEs are derived from the Hessian matrix of the log-likelihood function.

In the code, to the training dataset a column of ones have been added (with *np.stack* from NumPy) in order to create a design matrix that includes the intercept. The *y_pred* predicted probabilities have been used to compute the diagonal matrix *W*, defines as:

$$W = \text{diag}(\hat{y}(1 - \hat{y}))$$

The Hessian matrix is:

$$H = X^T W X$$

stabilized with a small epsilon to ensure invertibility, since the covariance matrix of the estimated parameters is the inverse of the Hessian:

$$\text{Cov}(\hat{\beta}) = H_{\epsilon}^{-1}$$

Finally, the SEs are the square roots of the diagonal elements.

$$SE(\hat{\beta}_i) = \sqrt{[\text{Cov}(\hat{\beta})]_{ii}}$$

The z-scores for each coefficient measure how many standard deviations the estimate is away from zero. They are computed as:

$$z_i = \frac{\hat{\beta}_i}{SE(\hat{\beta}_i)}$$

The p-values, instead, is the probability of observing a coefficient as extreme as beta under the null hypothesis $\beta = 0$.

$$p_i = 2 \left(1 - \Phi(|z_i|) \right)$$

Finally, to evaluate the goodness of the fit, we compute the Akaike Information Criterion, defined as:

$$AIC = 2k - 2 \sum_{i=1}^n \log L_i$$

where k is the number of estimated parameters (weights + the intercept), n is the number of observations and the log loss is the average log-loss on the training dataset.

-*Summary*: The summary method returns a table of estimated coefficients (including the intercept) and corresponding inferential statistics, formatted similarly to R's *glm* output. In the input, features names must be manually inserted, otherwise the algorithm will use default names automatically created. This function is supported by the *safe_format*, which guarantees that all valid floats are displayed consistently with four decimal places.

-*Predict*: The predict function applies the learned weights and bias to a linear combination of features, passes the result through the sigmoid function, and outputs probabilities.

3.2 Support Vector Machine

The idea behind Support Vector Machine is to find the optimal hyperplane that separates data points of different classes with the maximum margin.

The margin is defined as the distance between the hyperplane and the nearest points of each class, known as support vectors.

SVM can handle linear and nonlinear data by applying kernel functions, which implicitly map the input features into a higher-dimensional space where a linear separation becomes possible.

In the implementation through the SupportVectorMachine class goes through the following parameters: -*C*: regularization term.

-*lambda_reg*: automatically initialized as $1 / C$.

-*kernel*: linear, polynomial or radial.

-*probability*: the prediction output can either be in probabilities or in labels. This parameter is boolean and default is True.

-*gamma*: used for both polynomial and radial kernels.

-*d*: degree of the polynomial kernel.

-*r*: coefficient of the polynomial kernel:

-*epochs*: number of gradient descent iterations.

-*early_stopping*: boolean flag for stopping early based on validation loss.

-*max_iter*: maximum iterations without improvement for early stopping.

-*verbose*: prints training information if True.

3.2.1 Key functions

-*Sigmoid*: The logistic function transforms the linear combination of features into a probability in the interval $[0, 1]$. The sigmoid function chosen (expit) belongs to the package *scipy.special* and it designed to handle eventual numerical overflows.

Within the SVM it is function that does not have real importance other than allow the class to produce an output between 0 and 1.

-*Gamma computation*: gamma calculation. It can be computed automatically through *scale* and *auto* options, or can be set directly by the user as a numeric value.

When *auto*, gamma is computed as:

$$\gamma = \frac{1}{\text{number of features}}$$

When *scale*, the number of features is multiplied by the mean of the standard deviation of each feature, with a debugging feature that replaces 0 variance with 1.

-*kernel matrix creation*: the kernel matrix represents the inner product in the feature space, allowing SVM to operate in potentially higher-dimensional spaces without explicitly mapping the data.

The input accepts X1 and initializes X2 as None to compute the self-kernel matrix.

If the kernel chosen is polynomial, the standard dot product is computed and then polynomial kernel is applied:

$$K_{poly}(x_i, x_j) = (\gamma \langle x_i, x_j \rangle + r)^d$$

where gamma is the scaling factor, r is the constant term and d is the degree of the polynomial.

The radial kernel, instead, computed the squared norms of X1 and X2, then the squared Euclidean distance between each pair of points is calculated using broadcasting. Finally, the radial kernel is applied:

$$K_{RBF}(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

-*Hinge loss*: the hinge loss is the SVM loss function, measuring how well the decision boundary separates the classes. In the linear case, the loss for each training sample is computed as:

$$\text{margin}_i = y_i(\mathbf{w}^\top \mathbf{x}_i + b)$$

$$\ell_{hinge}^{linear} = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - \text{margin}_i)$$

As noticed, if the point does not have a margin equal or less than 1, the loss is just 0. The points that affect the result are the ones that lie on the margin (the support vectors). If regularization is applied, a penalty proportion of $\frac{\lambda}{2} \|\mathbf{w}\|^2$ is added to prevent large weights.

Kernel loss, instead, is computed in the dual space using the kernel matrix K and the Lagrange multipliers α . The decision value for each sample is

$$\sum_{j=1}^n \alpha_j K(\mathbf{x}_i, \mathbf{x}_j)$$

The regularization term is added as $\frac{\lambda}{2} \alpha^\top K \alpha$.

-*PEGASOS*: the Primal Estimated sub-Gradient Solver for SVM is a stochastic gradient descent algorithm designed to efficiently solve the SVM optimization problem. Instead of computing the gradients over the entire dataset, the PEGASOS randomly samples a single training example (a mini batch) at each iteration. This leads to faster convergence. The learning rate decreases over iterations.

PEGASOS function for the linear kernel first picks a random training example (xt and yt), computes the margin through the previously stated formula and updates the η parameter (the learning rate as $1 / \text{lambda_reg} * t$ in case the margin is less than 1).

The same happens with the PEGASOS function designed for kernel applications, which, instead, choses an index to update a Langrange multiplier. The margin is computed with the Kernel matrix column $K[:, \text{index}]$, and updates follow the same rule.

-*Fit function*: the fit method accepts training features and target, shuffles the dataset to prevent learning order patterns, and optionally creates a validation set for early stopping.

If polynomial or radial kernels are initialized, gamma is computed based on the input received; subsequently, parameters are initialized (weights for the linear kernel and positive multipliers for the polynomial and radial ones).

At the end of the algorithm, the fit function identifies and saves the support vectors, which are the critical points that define the decision boundaries.

In linear kernels, these points directly influence the orientation and position of the hyperplane: weights and bias are essentially determined by them.

In the other two cases, the support vectors are the points in which the Langrange multipliers are greater then zero.

The SVM only needs these points to make decisions, encapsulating the most informative samples that define the class boundaries.

-*Summary*: The summary method returns a table of estimated coefficients (including the intercept) and corresponding inferential statistics, formatted similarly to R's *e1071* output.

-*Predict*: once the best parameters are stored, they are used to make predictions. For a linear SVM, the model relies on the weights and bias; for the kernel SVM,

it takes support vectors into consideration.

$$\sum_{i \in SV} \alpha_i y_i K(x_i, x)$$

The function first computes the kernel matrix between the test points and the support vectors, measuring the similarity of each test point to the key points defining the decision boundary. The decision values are then obtained by weighting the contributions of the support vectors with their Lagrange multipliers and labels.

It returns the type of kernel applied and all the parameters values, plus the number of support vectors found.

4 Evaluation Metrics

The evaluation framework is encapsulated in a dedicated class, `Metrics`, which is designed to compute standard performance measures for binary classification models. The class accepts the following parameters:

-*y_true*: the true labels of the target variable.

-*y_pred*: the predicted outputs of the model as probabilities.

-*threshold* (default is 0.5): the probability threshold used to convert continuous predictions into binary labels for certain metrics.

Upon initialization, the class automatically converts predicted probabilities into binary predictions using the *binary_ypred* method and initializes internal attributes for key metrics (precision, recall, specificity, F1 score, ROC curve, and AUC) via the *_init_metrics* method.

The *acc_score* method computes the classification accuracy by comparing the binary predictions against the true labels, returning the mean accuracy over all samples.

The *conf_matrix* method calculates the confusion matrix along with precision, recall, specificity, and F1 score. The results are formatted as a tuple with a precision of four decimal places for clarity.

For model discrimination analysis, the *roc* method computes both the Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC). The ROC curve describes the model's ability to distinguish between positive and negative classes, with the True Positive Rate (TPR) defined as:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

The AUC is obtained by integrating the ROC curve using the trapezoidal rule. Thresholds for binary classification are generated via *np.linspace*, and TPR/FPR values are calculated for each threshold.

The *plot_roc* method visualizes the ROC curve. If the ROC has not been previously computed, it is generated automatically. The plot highlights two key points:

- Current threshold: the point corresponding to the user-defined threshold (default 0.5).

- Optimal threshold: the threshold minimizing the Euclidean distance from the ideal point (FPR=0,TPR=1), representing the best trade-off between sensitivity and specificity.

This structured design ensures a full evaluation of binary classification models, supporting both numerical assessment and visual inspection of predictive performance.

5 Hyperparameter tuning

In order to select the best model and hyperparameters, we split the training data into multiple folds and iterate over different parameter combinations. This process is called *Cross Validation* and has been implemented using a *GridSearch* function.

5.1 Stratified Cross Validation

First, the X and Y sets are converted into NumPy arrays for efficient manipulation. To ensure each folder preserves the original class distribution - a critical requirement for imbalanced datasets - the data is first sorted by class labels. Using *np.unique* and *np.cumsum*, the function determines the start and end indices for each class in the sorted arrays. This allows the data for each class to be split separately into K stratified folds. For each fold, one subset is reserved for validation while the remaining k-1 subsets form the training sets. For each iteration, the model is copied to avoid altering the original object, then it is trained and evaluation metrics are computed. The accuracy scores are stored in a list called *accuracy_scores*, which is initialized before the loop. The function returns the average accuracy across all folds.

5.2 GridSearch

This function performs hyperparameter tuning through cross-validation. It accepts a parameter grid (which must be provided as a dictionary or a list of dictionaries), initializes the variables *best_params*, *best_model*, and *best_accuracy*, and then generates all possible combinations of parameters using the product function from Python's *itertools* package. For each combination, a dictionary is created with parameter names as keys and the corresponding values. The model

is then initialized based on the class provided in the *model_class* argument using the current parameter combination. Cross-validation is used to evaluate the performance of each set of hyperparameters. The combination yielding the highest accuracy is stored along with *best_params* and *best_model*. Once the best model is identified, the best performing parameters are selected based on the highest average accuracy. It is important to notice that the GridSearch function does not train the final model on the full dataset.

6 Data Exploration

The dataset selected for this study is the Wine Quality dataset, which comprises chemical measurements of red and white wines, alongside quality ratings assigned by wine experts. The primary objective is to predict wine quality based on these physicochemical features:

- Fixed Acidity*: Concentration of acids such as tartaric acid.
- Volatile Acidity*: Amount of acetic acid, contributing to a vinegar-like taste.
- Citric Acid*: Naturally occurring acid enhancing freshness.
- Residual Sugar*: Sugar remaining post-fermentation.
- Chlorides*: Chloride concentration affecting flavor.
- Free Sulfur Dioxide*: Sulfur dioxide in free form, acting as a preservative.
- Total Sulfur Dioxide* : Sum of free and bound sulfur dioxide.
- Density*: Wine density.
- pH*: Acidity level of the wine.
- Sulphates*: Natural sulfur compounds that act as preservers of the microbiological fauna of wine, making it denser
- Alcohol*.

The target variable, wine quality, is originally scored on a scale from 0 to 10. For the purposes of this analysis, the variable was binarized, assigning 0 to scores 5 and 1 to scores 6. The application of the *np.unique* function allowed us to spot imbalance between categories (1640 for the negative class and 3258 for the positive one).

Prior to model development, a comprehensive correlation analysis was conducted. Given that all explanatory variables are numerical, Pearson’s correlation coefficient was employed. The analysis revealed moderate inter-variable correlations, including a positive correlation of 0.84 between residual sugar and density, and negative correlations of -0.78 between alcohol and density. In addition, moderate correlation (between 0.20 and 0.60) has been spotted among several variables, leading to considering to apply a model with reduced features. Principal Component Analysis (PCA) performed on the standardized dataset indicated that the first two principal components accounted for approximately 45% of the cumulative variance, suggesting that additional dimensionality reduction was not required.

To mitigate the influence of extreme values, outliers were addressed using the *winsorize* function from the *scipy.stats.mstats* library, ensuring robustness in subsequent analyses.

Finally, the relationship between the binary target variable and the explanatory features was examined using the point-biserial correlation (`pointbiserialr` from *scipy stats*). Results indicated either weak or negligible correlations, supporting the need for predictive modeling rather than simple feature selection based on correlation alone.

7 Models fitting and results

7.1 Data preprocessing

Prior to model development, the dataset was manually oversampled due to the high imbalance between the two classes constituting the target variable.

After identifying the majority and minority class based on their distribution, additional samples of minority class were generated by duplicating existing observations until the class frequencies were approximately equal.

Even though both the X and Y sets were converted into NumPy arrays, a copy of the oversampled explanatory variables dataset has been kept in order to facilitate variables selection.

The X NumPy array dataset was divided into a training set and a test set, and the explanatory variables were separated from the target variable. The features were then standardized using the `fit_transform` function from the *sklearn* package, while the column names were stored in a variable called `_names` to facilitate interpretation of the summary statistics.

Feature scaling is an essential preprocessing step that should not be overlooked before fitting any model. Since each variable is measured on its own scale, variables with larger magnitudes could disproportionately influence the model, leading to biased or misleading weight estimations. Standardization addresses this issue by centering each observation relative to the mean of its feature and scaling by the standard deviation. As a result, all features are transformed to have a mean of 0 and a standard deviation of 1, ensuring that the model evaluates each variable based on its relative importance rather than its unit of measurement.

7.2 Logistic regression results

The first model to be fit has been the logistic regression, at first with all the variables, subsequently with a reduced version of the dataset.

Hyperparameter optimization resulted in the following choice: learning rate 0.01, no regularization (even though in the GridSearch output penalty is set to 'L2'; if λ is not more than 0, regularization is not applied) and no early stopping. Several predictors were statistically significant, including volatile acidity (negative effect), residual sugar, free sulfur dioxide, density, sulphates and alcohol. The reduced model is based on a selection done on p-values, achieving a slightly lower accuracy. These results are consistent with the correlation analysis: alcohol, which showed the strongest positive correlation with wine quality, emerged

as the most important predictor in both models. Same for volatile acidity. Residual sugar, although weekly negatively correlated with quality, displayed a significant positive coefficient in the regression model. Sulphates also proved important in both models.

7.3 Support Vector Machine results

During the fitting of the Support Vector Machine (SVM) model, a limitation emerged in the hyperparameter tuning process. The cross-validation and grid search framework employed was designed to return only the model with the highest accuracy, which is not always the optimal choice for SVM. Unlike linear models, SVMs are highly sensitive to kernel selection and parameter settings, and the model that achieves the best training accuracy does not necessarily yield the best generalization to unseen data. This is particularly relevant due to the bias–variance trade-off: complex kernels, while capable of capturing nonlinear decision boundaries, are prone to overfitting if tuned solely for accuracy. To address this issue, instead of relying on a single global grid search, I tuned the hyperparameters separately for each kernel type (linear, polynomial, and radial basis function). This approach ensured a fairer comparison between kernels, allowed each to be optimized within its own parameter space, and ultimately facilitated the identification of models that balance both accuracy and generalization performance.

The linear kernel achieved an accuracy of 0.70 approx with an AUC of 0.77, showing balanced precision and specificity but lower recall, indicating that it tended to misclassify a notable portion of positive cases.

The polynomial kernel (degree 2) despite its ability to nonlinear boundaries, performed the worst. In contrast, the radial basis function delivered the best performance, with an accuracy rising to 0.89 and an AUC of 0.94.

An interesting outcome of SVM tuning was that, for both polynomial and radial basis function kernels, the optimal parameters search returned a regularization parameter $C = 0$. In practice, such a value corresponds to an extremely constrained margin, essentially forcing the classifier toward underfitting by penalizing all misclassifications heavily. This behavior suggests two things:

- 1) a limitation in the tuning process: the grid search was biased toward parameter values that maximized cross-validated accuracy in the training folds, but not necessarily those that generalized best;
- 2) it can be interpreted in light of data structure: C close to 0 produces a simple boundary, meaning that the feature space is linearly separable.

7.4 Predictions on new data and conclusions

Once the models have been trained, a new dataset has been loaded. While the training dataset contained information on white wines, the second ones focuses on red wine. The number of samples is definitely less and the target variable is almost perfectly balanced.

The results illustrate the generalization issues discussed earlier. While previously the radial kernel achieved excellent performance on the original dataset, on the new dataset the same configuration performed poorly, with an accuracy dropped to 60% and AUC to 50%. This demonstrates that the hyperparameters tuned to maximize accuracy on the original data did not generalize well to unseen data, highlighting the risk of overfitting inherent in SVMs with flexible nonlinear kernels.

Similarly, the polynomial kernel, which had already shown signs of underfitting previously, also failed to generalize, achieving moderate accuracy but poor recall and AUC.

In contrast, the linear models—both logistic regression and linear SVM—remained robust, maintaining comparable accuracy and AUC across datasets. These observations confirm that overly complex models may perform exceptionally on training or validation folds but fail to generalize to new samples, whereas simpler linear models tend to be more stable, supporting the earlier point that separate kernel tuning and cautious parameter selection are crucial for reliable generalization.